46. ML-Adjacent Pipelines: ETL, Batching, Features

46. ML-Adjacent Pipelines: ETL, Batching, Features

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `46-ml-pipelines`

Learning objectives

- Build ETL stages as interfaces
- Batch records for throughput
- Validate feature schemas
- Make pipelines restartable

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

ML pipelines in Go are often ETL, batching, and feature assembly before a model API. Reliable batching reduces cost and latency variance.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Batch records to amortize model call overhead.
- Deterministic feature transforms with tests.
- Backpressure when downstream is slow.
- Schema discipline: version your features.
- Idempotent job processing for retries.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-ml-adjacent && cd /tmp/gw-ml-adjacent
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func Batch(xs []int, size int) [][]int {
    var out [][]int
    for i := 0; i < len(xs); i += size {
        j := i + size
        if j > len(xs) {
            j = len(xs)
        }
        out = append(out, xs[i:j])
    }
    return out
}

func main() { fmt.Println(Batch([]int{1, 2, 3, 4, 5}, 2)) }
```

Run it:

```
go run .
```

Expected output:

```
[[1 2] [3 4] [5]]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement Batch.
2. Add a feature hash function for strings.
3. Simulate a worker that processes batches from a channel.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Pipeline stages

```go
type Stage[In, Out any] interface {
    Process(ctx context.Context, in In) (Out, error)
}
```

Compose extract → transform → load. Each stage should be testable offline.

Batching

Micro-batches improve throughput to model APIs and DBs. Flush on size or time (`time.Ticker`).

Features

Document types/units. Version feature schemas. Reject bad rows early with metrics.

Idempotent loaders let you replay safely after failures.

Failure modes to watch while coding

- Giant unbounded buffers
- Schema-free JSON forever
- Silent dropping of bad rows without metrics

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- pandas batching vs slice batching.
- Airflow/Dagster orchestration vs Go workers.
- Same need for deterministic transforms.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Log batch sizes and latency histograms — they diagnose model cost issues.

Practice checklist

Before the exercise, confirm you can:

1. Explain “ML-Adjacent Pipelines: ETL, Batching, Features” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Giant unbounded buffers
- Schema-free JSON forever

- Silent dropping of bad rows without metrics

Exercises

Work in `exercises/46-ml-pipelines/`. Read that folder's `README.md` before you code.

What you will build

Implement batching helpers per tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/46-ml-pipelines/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why batch?
2. What makes a pipeline restartable?
3. Stage interface benefit?

Further reading

- https://go.dev/blog/pipelines